

SAAR

Speech-Aware Adaptive Automation & Replay

Samsung PRISM Hackathon | Theme 03: Teachable Voice Automation

Organisation: Lesgo-HQ | Repository: github.com/Lesgo-HQ/SAAR

Team: Aashi Soni · Sourav Kumar · Ayush Kr Sharma · Anish Mall

1. Problem Statement

Voice assistants today cannot act inside third-party applications such as Amazon, Zomato, or Domino's — anything beyond a first-party feature falls back to a plain web search. Users already know exactly which taps get a job done; they simply have no way to transfer that procedural knowledge to an assistant. SAAR closes this gap: a user demonstrates a task once, by voice and touch, and the system learns a reusable, generalised flow it can replay on later spoken commands — including paraphrases and changed parameter values.

2. Objective — What SAAR Must Do

- Let a user speak a command, then record the on-screen steps as they perform it once.
- Generalise that single recording into a reusable flow, not a brittle literal tap-replay.
- Match later utterances — including paraphrases and changed slot values — to the learnt flow.
- Replay reliably even when screens change, and ask the user only when genuinely stuck.

3. Scope & Hard Constraints

These are evaluation-critical. Violating any of them caps or zeroes the corresponding score (the credential-capture rule carries an explicit −10 penalty).

- **Platform:** Android only. Automation must go through Accessibility Service / UI-Automator-style APIs. No app-specific SDKs, no deep links used as a substitute for taps, no web fallbacks.
- **No hard-coded flows:** judges will teach a brand-new flow live during evaluation. Anything that only works on pre-baked scripts scores zero on generalisation (test cases T2–T9).
- **No credential capture:** the system must detect and pause on any payment, OTP, password, or login screen, and hand control back to the user.
- **Parametrised flows:** a flow taught as “order a Margherita pizza from Domino’s” must generalise to “order garlic bread from Domino’s” without being re-taught.
- **Condition-change handling:** the system must detect a changed state (promo pop-up, pre-filled cart, language change, logged-out session) and either proceed sensibly or ask for confirmation.
- **Coverage:** at least two or three third-party apps, and at least one parametrised slot (item, quantity, or address) is the minimum bar to prove the idea.
- **Bonus – noise filtering (+3):** discard incidental actions captured during teaching that are not part of the task (e.g. dismissing an incoming call).
- **Bonus – cross-app generalisation (+4):** a flow learnt on one app should transfer to a structurally similar app (Amazon → Myntra).
- **Bonus – mid-flow clarification (+3):** recognise a missing/ambiguous parameter mid-flow, ask, and continue rather than failing or re-teaching.

3.1 Interpretation — what this rules out

Any design built around fixed selectors, recorded coordinate taps, or per-app integration code fails the “no hard-coded flows” test by construction, because a judge-taught flow is guaranteed to reference UI elements and text the system has

never seen. The only architecture that survives live teaching is one where the flow is learnt as an *abstract, parametrised plan* over UI element roles and semantics, not literal screen coordinates or literal spoken text. This single constraint is what drives most of the technology choices in Section 5.

4. Submission Deliverables (from evaluation brief)

- Installable APK.
- Source code repository: github.com/Lesgo-HQ/SAAR.
- Demo video, ≤ 5 minutes, unedited and strictly in this order: (a) teach one flow by voice + taps, (b) replay with the exact utterance, (c) replay with a paraphrase, (d) replay with a changed slot value, (e) the assistant asking a question when genuinely stuck.
- Architecture write-up covering speech-to-intent, UI-tree capture, generalisation, slot extraction, and replay, with diagrams.
- Target-app declaration.
- Known limitations.

5. System Architecture

SAAR is organised as a pipeline of eight cooperating modules, all running on-device except the reasoning calls to an LLM API. The design principle throughout: **never store or match on literal screen text or coordinates** — always store on element *role/semantics*, so the same flow definition survives paraphrase, slot change, minor UI change, and (for the cross-app bonus) a different app entirely.

#	Module	Responsibility
1	Voice Capture & ASR	Continuous/push-to-talk capture; converts speech to text.
2	Speech-to-Intent (NLU)	Classifies utterance as Teach / Command / Query; extracts intent and candidate slot values.
3	UI-Tree Capture	Accessibility Service listens to window/content-changed events and snapshots the AccessibilityNodeInfo tree (role, text, content-description, resource-id, bounds) at each user action.
4	Teaching & Flow Synthesiser	Converts one raw (utterance + action-trace) demonstration into an abstract, parametrised flow definition; filters incidental/non-task actions (bonus).
5	Flow Matcher	Given a new utterance, retrieves the closest learnt flow via semantic similarity and confirms intent, quantity, and other slots.
6	Replay & Adaptation Engine	Executes a matched flow step-by-step through the Accessibility Service; on each step, diffs the live UI tree against the expected state and adapts, retries, or escalates.
7	Credential / Sensitive-Screen Guard	Runs before every action; heuristically and semantically detects payment, OTP, password, and login screens and force-stops automation there.
8	Clarification & Reporting	Asks a targeted question when the matcher is ambiguous or the replay engine cannot resolve a state; logs step-level success/failure for the reporting test case.

5.1 Execution flow

Teach: Voice Capture → Speech-to-Intent (mode = Teach) → UI-Tree Capture records every user action as (screen-state, action, target-node) → Flow Synthesiser abstracts the trace into a parametrised flow → stored via the Storage Layer.

Replay: Voice Capture → Speech-to-Intent (mode = Command) → Flow Matcher selects a flow and resolves slots → Replay Engine executes steps against the live UI tree, checking the Credential Guard before every tap → on divergence, attempt local adaptation, else ask via Clarification module → Reporting logs the outcome.

5.2 Flow representation (illustrative schema)

```
{
  "flow_id": "domino_order_pizza",
  "app_package": "com.dominos.app",
  "trigger_intent": "order_food",
  "example_utterances": ["order a margherita pizza from domino's"],
  "slots": [
    {"name": "item", "type": "string", "default": "Margherita"},
    {"name": "quantity", "type": "int", "default": 1},
    {"name": "address_label", "type": "enum", "values": ["Home", "Work"]}
  ],
  "steps": [
    {"id": 1, "action": "tap", "target_role": "search_box"},
    {"id": 2, "action": "type", "target_role": "search_box", "value_slot": "item"},
    {"id": 3, "action": "tap", "target_role": "first_search_result"},
    {"id": 4, "action": "tap", "target_role": "add_to_cart_button"},
    {"id": 5, "action": "set_quantity", "value_slot": "quantity"},
    {"id": 6, "action": "select", "target_role": "delivery_address", "value_slot": "address_label"},
    {"id": 7, "action": "stop_before", "target_role": "payment_screen"}
  ]
}
```

The key design decision is **target_role, not resource-id or literal text**: roles are inferred at teaching time from a small ontology (search box, result card, add-to-cart, quantity stepper, address selector, pay button, OTP field, etc.), which is what makes slot substitution, paraphrase matching, and the cross-app bonus possible without re-teaching.

6. Technology Stack & Justification

Each choice below is the one we commit to, with the primary alternative that was considered and why it was rejected for this brief specifically — not in general.

Layer	Choice	Why (vs. main alternative)
Automation API	Android AccessibilityService	UiAutomator is an instrumentation/testing API driven over ADB from a host machine — it cannot ship inside a normal installed app acting autonomously on-device, which the brief requires. AccessibilityService is the only API that lets a shipped app read another app's UI tree and dispatch real gestures at runtime, matching the letter of the constraint.
Language / platform	Kotlin, native Android	Direct AccessibilityService and gesture-dispatch access; cross-platform frameworks (Flutter/React Native) add a bridging layer to the exact APIs this project lives or dies on, for no benefit since scope is Android-only.
Speech-to-text	On-device ASR (Android SpeechRecognizer / offline recognizer) as primary, cloud ASR (e.g. Google Cloud Speech-to-Text) as optional fallback	Live judging happens on venue Wi-Fi; an ASR call that depends on network is a demo-day failure risk. On-device recognition removes that single point of failure. Cloud ASR is kept only as an optional accuracy fallback, not the primary path.
Intent + slot extraction	LLM-based (hosted, small/fast model class) over a rule engine or classic NLU (Rasa/Dialogflow)	Judges teach an unseen flow live and test paraphrases (T3) — a fixed intent grammar or trained classifier cannot generalise to utterances it never saw. An LLM given the utterance plus the current UI context can extract intent and slots zero-shot, which is the only approach that survives the “no hard-coded flows” rule.
Flow synthesis (teach → abstract plan)	LLM reasoning over the captured action trace + UI tree, producing the JSON flow schema in §5.2	This is a program-synthesis problem (concrete demonstration → generalised, parametrised plan). Hand-written heuristics would need a special case per app; an LLM given structured node data performs this abstraction directly and is what makes the bonus (noise filtering, mid-flow clarification) tractable in hackathon time.
Utterance-to-flow matching	Sentence-embedding similarity (top-k retrieval) + LLM re-rank/verify on the shortlist	Calling the LLM to classify against every stored flow does not scale and adds latency on every command. Embeddings give near-instant candidate retrieval; the LLM is invoked only to verify/disambiguate the top few candidates, which keeps replay latency low without sacrificing paraphrase robustness.
Replay-time adaptation	UI-tree diff/hash against expected state, with an LLM decision step (dismiss / adapt / ask) invoked only on divergence	Pure literal tap-replay breaks the instant a popup or pre-filled cart appears (T7). Full LLM-in-the-loop on every single step is unnecessarily slow and costly. Diffing locally and escalating to the LLM only on divergence is the latency/robustness balance the test suite is explicitly designed to reward.
Local storage	Room (SQLite) for flow definitions + session logs; in-memory vector index for embeddings	The flow count in a hackathon demo is small (a handful of taught flows). A managed vector database is unnecessary operational overhead; Room plus an in-memory cosine-similarity index is simpler, offline-safe, and sufficient at this scale.

Layer	Choice	Why (vs. main alternative)
Credential / sensitive-screen guard	Rule-based screen classifier (resource-id/content-description patterns for password/OTP/payment fields, `InputType` flags) checked before every dispatched action	This check must be deterministic and fail-closed, since a miss carries a −10 penalty — it is the one place in the system that should <i>not</i> depend on a probabilistic LLM call. Rule-based detection on Android's own input-type and autofill hints is reliable and auditable.

Note on the LLM dependency: every LLM-in-the-loop step above is a genuine network dependency. Mitigate demo risk by caching the synthesised flow and the embedding index locally after teaching, so only the teach step and true ambiguity/adaptation cases need a live call — exact and slot-substituted replay (T2, T4–T6) can run primarily off the cached local plan.

7. Test-Case Traceability

Mapping the evaluation suite to the module(s) that must work correctly for that case to pass. Use this as the build order — modules feeding the earliest, highest-weighted test cases come first.

ID	Case	Primary module(s) responsible	Pts
T1	Teach (food)	UI-Tree Capture, Flow Synthesiser	5
T2	Exact replay	Flow Matcher, Replay Engine	5
T3	Paraphrase (x2)	Speech-to-Intent, Flow Matcher (embeddings)	6
T4	Slot: item	Flow Matcher (slot resolution), Replay Engine	4
T5	Slot: quantity	Slot resolution	4
T6	Slot: address	Slot resolution	4
T7	Screen change	Replay/Adaptation Engine (UI diff + LLM decision)	6
T8	Teach (e-commerce)	Flow Synthesiser (second, distinct flow)	4
T9	Cross-app slot + replay	Flow Matcher, Replay Engine (generalised search term)	4
T10	Genuinely stuck	Replay Engine + Clarification module	5
T11	Credential boundary	Credential/Sensitive-Screen Guard	5 (fail = -10)
T12	Negative/unknown intent	Flow Matcher (confidence threshold)	3
T13	Ambiguity	Flow Matcher, Clarification module	2
T14	Reporting	Reporting/logging	3

8. Build Plan — Phased Milestones

Phase 0 — Foundation

- Android project scaffold; AccessibilityService with node-tree capture and gesture dispatch verified on one target app.
- On-device ASR wired to a push-to-talk trigger; raw utterance reaching the app.
- Target apps installed and manually mapped (element roles for search, results, add-to-cart, quantity, address, pay/OTP screens).

Phase 1 — Teach & exact replay (T1, T2, T11)

- Flow Synthesiser producing the JSON schema from one recorded demonstration.
- Credential/Sensitive-Screen Guard implemented and unit-tested first — this gate blocks everything downstream and carries the -10 penalty.
- Literal replay of a stored flow working end-to-end on the first target app.

Phase 2 — Generalisation (T3–T6, T8, T9)

- Embedding-based Flow Matcher; paraphrase test utterances passing.
- Slot resolution for item, quantity, address.
- Second flow taught on a second app; cross-app search-term generalisation attempted.

Phase 3 — Robustness (T7, T10, T12, T13, T14)

- UI-diff based divergence detection; LLM-backed adapt/ask decision on divergence.
- Stuck-state detection (logged-out, language change) with a bounded 30-second decision window.
- Unknown-intent rejection and ambiguity clarification; session-level success/failure reporting.

Phase 4 — Bonus & polish

- Noise-touch filtering during teaching.
- Cross-app UI generalisation beyond the search-term case (Amazon → Myntra style transfer).
- Mid-flow parameter clarification (missing restaurant/parameter mid-flow).
- Demo video in the mandated shot order; architecture diagrams; known-limitations write-up; APK build.

8.1 Suggested role split (4 members)

- **Automation core** — AccessibilityService, node-tree capture, gesture dispatch, Credential Guard.
- **Voice & language** — ASR integration, Speech-to-Intent, embedding-based Flow Matcher.
- **Flow intelligence** — Flow Synthesiser, replay-time adaptation/LLM decision loop, slot resolution.
- **Product & QA** — Storage layer, Clarification/Reporting UX, target-app mapping, test-case rehearsal against the exact judge script, demo video, architecture write-up.

This is a starting split, not an assignment — reallocate against actual strengths in the team; the four tracks above should still run in parallel from Phase 1 onward.

9. Target-App Declaration (proposed)

Minimum bar is two to three apps. Proposed set, chosen because they appear directly in the judge's test script (T1–T10) and give one food-delivery pair for cross-app comparison plus one e-commerce app:

- Domino's (food ordering, primary teach target).
- Zomato (food ordering, second food app for cross-app/alternate-flow coverage).
- Amazon (e-commerce, search + add-to-cart flow; Myntra as the bonus cross-app transfer target).

10. Known Limitations (to keep updated as built)

- LLM-in-the-loop steps (teach, ambiguity resolution, divergence adaptation) require network connectivity; these are the components most exposed to venue Wi-Fi conditions.
- Element-role inference is bounded by a fixed ontology (search box, result card, add-to-cart, quantity stepper, address selector, pay/OTP field); a UI pattern outside this ontology will not be recognised correctly on first exposure.
- Cross-app generalisation (bonus) is expected to work reliably only across structurally similar apps (e.g. Amazon-style catalog layouts); visually dissimilar apps are out of scope for the hackathon timeline.
- [Add empirically observed limitations here as testing proceeds — this section must reflect what was actually built, not only what was planned.]

11. Submission Checklist

- ■ Installable APK
- ■ Source repository: github.com/Lesgo-HQ/SAAR
- ■ Demo video ≤ 5 min, unedited, in mandated order (teach $\rightarrow$ exact $\rightarrow$ paraphrase $\rightarrow$ slot change $\rightarrow$ stuck/ask)
- ■ Architecture write-up + diagrams (this document + system diagram)
- ■ Target-app declaration (Section 9)
- ■ Known limitations (Section 10, kept current)